SMART TASK AND DEADLINE MANAGEMENT SYSTEM

Project Report

Subject: Computational Thinking and Programming

Implementation: Python, Flask, SQLite, HTML, CSS

Prepared as a project defence and demonstration submission

Abstract

The Smart Task and Deadline Management System is a web-based application for organizing tasks, deadlines, priorities, estimated durations, and prerequisite relationships. It provides task creation, editing, deletion, search, filtering, status tracking, dashboard metrics, overdue indicators, and a recommended dependency-aware schedule. The backend is written in Python using Flask, with SQLite persistence and HTML/CSS templates.

The scheduling engine applies Kahn's topological-sort algorithm to respect task dependencies and a heap-based greedy policy to choose among tasks that are currently eligible. The project demonstrates problem decomposition, data-structure selection, modular design, validation, unit testing, complexity analysis, and a command-line deadline report.

1. Problem Statement

Students and other users may manage assignments, personal work, and deadlines across separate notes or calendars. A scattered workflow can make it difficult to identify overdue work, compare priorities, and ensure prerequisite tasks are completed in the correct order. The proposed system centralizes task records and recommends a valid execution order.

2. Objectives

• Provide a web interface for task CRUD operations, searching, and filtering.

• Store deadlines, priority, estimated duration, status, and dependencies.

• Calculate dashboard counts, completion percentage, and overdue tasks.

• Generate a schedule that respects prerequisite relationships.

• Detect missing dependencies and circular dependencies.

• Use modular Python code and SQLite persistence.

• Provide automated tests, a reminder report, and complexity analysis.

3. Scope and Assumptions

The application is designed as a single-user student demonstration. It does not include authentication, cloud synchronization, email delivery, or multi-user permissions. The reminder script prints a report to the terminal and can be scheduled externally. The scheduler is a heuristic and is not guaranteed to find a globally optimal timetable.

4. Computational Thinking

4.1 Decomposition

The problem is decomposed into task management, data persistence, scheduling, dependency validation, dashboard metrics, and reminder reporting.

4.2 Abstraction

A task is represented by a small record containing an identifier, title, description, priority, deadline, estimated duration, status, and dependency identifiers. Database details are isolated in database.py, while algorithm logic is isolated in scheduler.py.

4.3 Pattern Recognition

Repeated task operations follow CRUD patterns. Prerequisite relationships form a directed graph. Selecting the next available high-priority task is a priority-queue pattern.

4.4 Algorithm Design

Kahn's algorithm finds a topological ordering. A heap chooses among eligible tasks by higher priority, earlier deadline, and then lower task ID. If not every task can be output, a cycle exists.

5. Functional Requirements

• Create tasks with required title, deadline, priority, estimated duration, and optional description.

• Edit task details and status; mark tasks complete or reopen them.

• Search by title or description and filter by status.

• Delete tasks and remove references to the deleted task from other tasks.

• Generate a schedule that places prerequisites before dependent tasks.

• Display dashboard totals, status counts, overdue count, and completion percentage.

• Print overdue tasks and tasks due within the next 24 hours from the reminder script.

6. Non-Functional Requirements

• Maintainable code through separate modules.

• Use parameterized SQL statements and form validation.

• Provide a responsive interface for desktop and smaller screens.

• Use automated tests for core scheduling and helper behavior.

• Keep local setup simple using Python, pip, and SQLite.

7. System Architecture

The browser sends requests to Flask routes. Routes validate input, call the persistence layer, and render Jinja templates. The persistence layer uses SQLite and parameterized queries. The scheduler module contains dependency parsing, topological ordering, priority selection, and helper functions. Tests exercise the scheduling and helper modules.

Layer	Responsibility
Presentation	HTML templates and CSS dashboard
Application	Flask routes, request validation, status updates
Algorithm	Dependency graph, topological sort, heap-based selection
Persistence	SQLite schema and CRUD functions
Quality	pytest, optional static analysis, GitHub Actions workflow

8. Data Model

The tasks table stores id, title, description, priority, deadline, estimated_minutes, status, dependencies, and created_at. Dependencies are stored as comma-separated task IDs to keep this educational project easy to inspect. A larger production system would typically normalize dependency relationships into a separate table.

9. Algorithms and Data Structures

9.1 Topological Sort

Each task is a vertex. A dependency from task A to task B means A must be scheduled before B. The algorithm calculates each task's incoming dependency count, adds tasks with zero incoming dependencies to a ready heap, repeatedly removes the best eligible task, and releases its dependents.

9.2 Greedy Selection

Among eligible tasks, the heap uses the tuple (-priority, deadline, task ID). This gives higher priority first, then earlier deadline, then a stable ID tie-break. Dependencies always take precedence over priority: a dependent task is not eligible until its prerequisites have been emitted.

9.3 Complexity

Let V be the number of tasks and E the number of dependency links. Graph construction is O(V + E). Heap insertions and removals contribute O(V log V), yielding O((V + E) log V) overall. Dictionary lookups are O(1) on average; task sorting is O(n log n). These are theoretical bounds, not measured benchmark results.

Operation	Data structure / method	Complexity
Task lookup	Dictionary / SQL primary key	O(1) average in a Python dictionary
Sort tasks	Comparison sort	O(n log n)
Heap push/pop	Binary heap	O(log n)
Topological ordering	Graph + heap	O((V + E) log V)
Completion rate	Single pass	O(n)

10. Object-Oriented and Functional Design

The project separates routes, database operations, and scheduling logic into modules. This separation is a practical form of modular design. Pure helper functions such as completion_rate and parse_dependencies can be tested independently. For a future expansion, task behavior could be encapsulated in domain classes, but classes are not required merely for the sake of using OOP.

11. Concurrency and Reminders

The reminder utility checks task deadlines and prints tasks that are overdue or due in the next 24 hours. It can be invoked periodically by cron or Windows Task Scheduler. Concurrency is not needed for the core local application because database operations are short and task counts are expected to be modest. Background workers or asynchronous processing would be an extension if the system later handles many independent notifications.

12. Testing Plan

• Dependency ordering: verify that every prerequisite appears before its dependent task.

• Priority ordering: verify that higher-priority eligible tasks are chosen first.

• Cycle detection: verify that a circular dependency raises an informative error.

• Missing dependency: verify that an unknown prerequisite is reported.

• Dependency parsing: verify whitespace handling and duplicate IDs.

• Completion percentage: test empty, partially completed, and fully completed lists.

• Overdue status: verify completed tasks are never shown as overdue.

Run the test suite with `pytest -q`. A test suite validates only the covered behavior; code coverage and end-to-end browser testing should be added during further validation.

13. Step-by-Step Implementation

Step 1 — Install Python

Install Python 3.10 or later and verify `python --version` in a terminal.

Step 2 — Extract the archive

Extract the ZIP file and open a terminal in the project directory.

Step 3 — Create a virtual environment

Run `python -m venv .venv`, then activate it using the command for your operating system.

Step 4 — Install dependencies

Run `pip install -r requirements.txt`.

Step 5 — Start the application

Run `python app.py`; open `http://127.0.0.1:5000` in a browser.

Step 6 — Create sample tasks

Add a task with a deadline and priority. Add a second task whose dependency field contains the first task's ID.

Step 7 — Inspect the schedule

Open Smart schedule. Confirm that the prerequisite appears before its dependent task.

Step 8 — Test task lifecycle

Edit a task, mark it complete, reopen it, search/filter tasks, and delete a task.

Step 9 — Run tests

Run `pytest -q` and review the output.

Step 10 — Run the reminder report

Run `python cron_reminders.py`; the script prints overdue tasks and tasks due within 24 hours.

Step 11 — Review quality

Review parameterized SQL, input validation, and the GitHub Actions workflow. Optional tools such as Ruff and mypy can be installed and run.

Step 12 — Prepare the defence

Demonstrate the dashboard, dependency-aware scheduling, cycle tests, complexity analysis, and reminder report.

14. Demonstration Chronology

• 0:00–1:00 — Introduce the problem statement, target users, and objectives.

• 1:00–2:00 — Explain task fields, database, and architecture.

• 2:00–4:00 — Create tasks and demonstrate priorities and deadlines.

• 4:00–5:00 — Demonstrate prerequisite scheduling and explain the heap.

• 5:00–6:00 — Show dashboard metrics, search, filtering, and overdue flags.

• 6:00–7:30 — Run automated tests and explain complexity.

• 7:30–8:30 — Run the deadline reminder report and discuss quality practices.

• 8:30–10:00 — Explain limitations, future work, and answer questions.

15. AI-Assisted Software Engineering

AI-assisted tools may be used to brainstorm requirements, draft implementation options, and review code. Generated code must be checked against the specification, run locally, tested with known examples and edge cases, and reviewed for unsafe assumptions. A useful defence record includes prompts, changes made after review, test output, and known limitations.

16. Limitations

• No authentication or multi-user access control.

• No email, SMS, or push notification integration.

• The scheduler is a heuristic, not a globally optimal planner.

• Dependencies are stored as comma-separated IDs for simplicity.

• The development server is not suitable for public production deployment.

• Actual performance results must be measured on the machine used for the demonstration.

17. Future Enhancements

• User accounts, roles, and shared task lists.

• Calendar views and recurring tasks.

• Email reminders and notification preferences.

• More scheduling policies and benchmark comparisons.

• A normalized dependency table, migrations, and stronger database constraints.

• Docker deployment with production-ready WSGI server and security configuration.

18. Conclusion

The Smart Task and Deadline Management System turns task organization into a practical programming project. It combines a working web interface with database persistence, task lifecycle operations, graph-based dependency validation, heap-based scheduling, automated tests, and a deadline report. It supports a meaningful technical defence because the algorithmic decisions and trade-offs can be demonstrated directly.

Appendix A — Project Files

• app.py — Flask routes and validation.

• database.py — SQLite schema and CRUD functions.

• scheduler.py — dependency parsing, priority scheduling, and helper functions.

- templates/ — dashboard, task form, and schedule pages.
- static/style.css — responsive interface.
- tests/test_scheduler.py — unit tests.
- cron_reminders.py — deadline report utility.
- IMPLEMENTATION_GUIDE.md — implementation steps.
- presentation.pptx — project defence slides.